Saga Pattern

long transactions as steps you can undo

Watch the 4-min video

In one line: a saga splits a multi-service transaction into local steps, each with a compensating action; if a step fails, the completed steps are compensated in reverse, coordinated either by an orchestrator or by services reacting to events, and every step must be idempotent and retryable.

1 Book a trip

A customer books a trip: a flight, a hotel and a rental car, from three different providers. The flight and hotel succeed. The car fails. You can't hold locks across three companies' systems for minutes, and they don't speak 2PC. What you can do is **cancel** the flight and the hotel. That's the saga idea, first described for long-lived database transactions in 1987.

2 Steps and compensations

A saga is a sequence of **local transactions** T1, T2, ... Tn. Each has a **compensating transaction** C1, C2, ... that semantically undoes it. If Tk fails, the saga runs Ck-1 ... C1 in reverse order. The outcome is all steps done, or all completed steps compensated. In between, other users can see partial states: sagas give up isolation.



Figure 1. The car fails, so the hotel and then the flight are cancelled.

3 Orchestration or choreography

Orchestration: one component (the orchestrator) tells each service what to do and tracks the saga's state; easy to see and change the flow. Workflow engines such as Temporal, AWS Step Functions or Camunda make the orchestrator durable. **Choreography:** each service listens for events and reacts (FlightBooked → book hotel); no central brain, but the flow is spread across services and harder to follow as it grows.



Figure 2. Orchestration centralises the flow; choreography distributes it through events.

WORKED EXAMPLE: A TRIP SAGA IN TEMPORAL

The workflow books the flight, then the hotel, then the car, registering a compensation after each success. The car API returns 'no availability'. The workflow runs the registered compensations in reverse: cancel hotel, cancel flight, retrying each with backoff until it succeeds. Each activity sends an idempotency key (`trip-512-flight`), so retries after a timeout never double-book. Because Temporal persists the workflow's history, a crash of the worker in the middle simply resumes the saga where it stopped.

4 Rules for good sagas

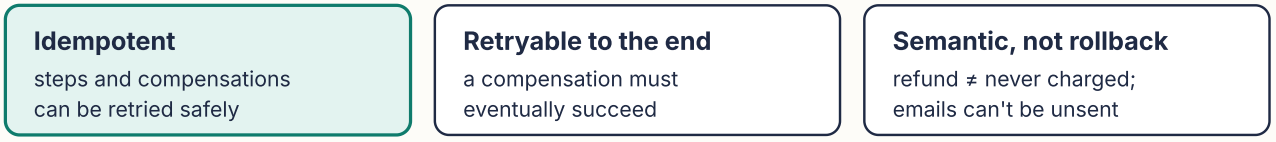


Figure 3. Compensations are business actions, not database rollbacks.

- Order steps so the ones most likely to fail, or hardest to undo, come first; put irreversible steps (send email, ship) last.
- Use **semantic locks**: mark records as PENDING so other flows know they're in progress.
- Make the saga's state durable (a workflow engine or a saga table), and publish events through an outbox.
- Plan for a compensation that can't succeed automatically: alert a human.

COMMON MISTAKES

- Compensations that aren't idempotent, so a retry refunds twice.
- Keeping saga state only in memory.
- Choreography across ten services with no way to see where a saga is stuck.
- Treating PENDING data as final in other features.

5 Key terms

Saga: sequence of local transactions with compensations

Orchestration: a central coordinator drives the steps

Semantic lock: a status flag marking work in progress

Compensating transaction: semantic undo of a completed step

Choreography: services react to each other's events

Pivot step: the point after which the saga only moves forward

6 Interview questions

Q1. What is a saga? BASIC

A way to run a multi-service transaction as a series of local transactions, each with a compensating action that's run in reverse if a later step fails.

Q2. Orchestration vs choreography? BASIC

Orchestration has a central coordinator that calls each step and tracks state; choreography has services react to events. Orchestration is easier to observe; choreography is more decoupled.

Q3. Design hotel and flight booking with a saga. SCENARIO

Orchestrator books flight then hotel with idempotency keys; on hotel failure it cancels the flight. Saga state is durable, steps are retried with backoff, and failures that can't be compensated alert a person.

Q4. A compensation keeps failing. What do you do? SCENARIO

Retry with backoff since it must eventually succeed; if it's permanently impossible, park the saga in a failed state and escalate for manual handling.

Q5. What anomalies can sagas cause? TRICKY

Lack of isolation: other transactions see intermediate states (dirty reads), lost updates if two sagas change the same data, and fuzzy reads. Use semantic locks and re-validation.

Q6. Why put irreversible steps last? TRICKY

Once something can't be undone, like shipping or an email, nothing after it may fail and require compensation, so it should be the final step.

Next: SD-H-018 · Eventual Consistency in Practice backendroadmap.com